Scope before Reuse: Risk-Controlled Admission of Reusable Code Changes

ANONYMOUS AUTHOR(S)

An agent that learns a code change once and reuses it everywhere is efficient until the change runs where it does not belong. The failure is functional, not a permission violation: an authorized change whose replay tests pass on its source can implement the wrong behaviour for the target, and two changes that are each beneficial can be jointly harmful. We formulate risk-controlled admission relative to a declared scope family and give Gatekeeper, which admits a change in a scope only when paired trials on that scope certify a benefit margin. We prove an anytime-valid guarantee that holds over a whole library, over adaptively chosen cells, over compositions and over re-admission after the environment changes, through a per-cell error ledger; we show that its shift term is necessary and tight, characterise composition risk exactly, and bound every executed set through an execution rule. Every statement is checked numerically. Pre-registered experiments, negative results included, show where such a gate matters. On 14 real skill-reuse targets harm was frequent and a simple gate with target evidence sufficed, while replay plus fingerprints admitted all four harmful cells. In a simulated library built on our implementation, admitting skills one by one ran a harmful composition in 28% of libraries, and composite cells with the execution rule in none; on a twelve-family library over 16 real repositories, footprints read off the original code missed a real interaction between two rewrites, which the execution rule now catches. In 205 plausible LLM patches, the test census admitted all 16 with partial behavioural differences and the statistical gates none, yet a manual check traced most of those differences to inputs the developers' fix also rejects; on SWE-bench Verified the measured harm rate changes by a factor of about four with its definition.

Additional Key Words and Phrases: code reuse, coding agents, e-processes, anytime-valid inference, software engineering

1 Introduction

Coding agents increasingly package what they learn as reusable procedures (skills) and apply them again elsewhere: a library of learned edits that fixed a bug once is replayed on the next repository [16, 21]. Reuse is efficient until a procedure runs where it does not belong. The failure is not a permission violation: an authorized skill that edits only files it may edit [9] can still implement the wrong behaviour for this repository, this dependency version, or this combination with another skill. Replay of the skill's own tests on its source repository [21] cannot see the target, and re-running the target's whole test suite before every reuse removes the reason to have skills. In our own experiments the target's suite missed harm that a skill caused on 87–100% of relevant inputs (Section 7.3).

We formulate *risk-controlled admission*: a change may execute in a declared scope only when paired evidence gathered in that scope certifies that it helps by at least a margin, with an error budget that holds over the whole library, over adaptively chosen cells, over compositions and over re-admission after the environment changes.

Contributions.

- **Formulation.** Selective transfer risk relative to a declared scope family, cells for single skills and for compositions, and an execution rule that ties per-cell certificates to what actually runs (Section 3).
- **Theory.** An anytime-valid, library-wide admission guarantee with a per-cell error ledger that covers compositions and re-admission (Theorem 5.2); a proof that its 2ξ shift term is necessary and tight (Proposition 5.4); an exact characterisation of composition risk, $\sup \mathrm{STR}(S_1 \circ S_2) = 1 + \min(a, b)$ (Theorem 5.5); and a bound on every executed set under a

stated non-interaction assumption (Corollary 5.6). Every statement is checked numerically in the artifact.

- **Gatekeeper.** An implementation with scope screening, an abstaining AST binder, paired trials in isolated workers, betting e-processes, a per-cell ledger and the execution rule (Section 6).
- **Evidence**, pre-registered and including negative results (Section 7): where simple gates fail; library-level control of compositions in a simulation built on the implementation, and a census of a twelve-family skill library on 16 real repositories that found an interaction footprints from the original code miss; skill reuse on 14 real targets against replay-plus-fingerprint and always-retest baselines; partial harm in 205 plausible LLM patches; and, on SWE-bench Verified, how much of the measured "harm" depends on its definition.

What we do not claim. We claim no new statistics, no advantage in statistical power over a fixed-sample test at the same number of trials (Section 7.1), no distribution-free guarantee on unseen repositories, and no results on large learned skill libraries: the admission studies use two hand-written rewrite families, and the twelve-family library is measured for overlap and interaction, not admitted.

2 Motivating Example

Consider an *illustrative* skill S_1 that wraps database calls in a bounded retry. It was learned on a repository whose client has at-least-once semantics; on a target whose client is exactly-once, the retry duplicates writes under partial failure. The skill is authorized (it edits application code only), its replay tests pass on the source, and the target's environment fingerprint can agree on every recorded variable while differing in the one that matters, delivery semantics, which nobody recorded. Only evidence gathered on the target can reveal the problem. Now add a skill S_2 that turns the call into a coroutine. If S_1 wraps the call before it is awaited, its retry "succeeds" on the coroutine object and the real failure is never raised: each skill is beneficial alone, the pair is harmful. Theorem 5.5 shows that nothing about the two individual risks prevents this, which is why compositions are cells of their own and why the execution rule refuses to run overlapping skills together until their composite is admitted.

3 Problem Formulation

Skills, scopes and cells. A skill is $S = \langle \pi, \mathcal{D}, \mathcal{I}, \mathcal{P}, \mathcal{K} \rangle$: a procedure π (a parameterised edit program), dependency and environment fingerprints $\mathcal{D}$, input assumptions $\mathcal{I}$ (executable predicates on the target), observed postconditions $\mathcal{P}$ with the contexts they were observed in, and an interaction footprint $\mathcal{K}$, the code regions and behavioural properties that π reads or writes. A context x is a (repository, revision, environment, task) tuple. A scope family $\mathbb{S} = \{s_1, \ldots, s_L\}$ is a finite partition of contexts declared before any trial. We call a pair $c = (S, s)$ a *cell*; for a set of skills whose footprints form a connected overlap graph, their composition, applied together in scope s, is a cell of its own (for two skills, $(S \circ S', s)$).

Loss and selective transfer risk. For a task x, $\ell(x, \pi) \in [0, 1]$ is its functional loss with the skill applied and $\ell(x, \varnothing)$ the loss without it. We use one minus the held-out pass rate plus 0.5 per regression, *clipped at* 1, so that the range assumption below holds. The paired difference is $Z = \ell(x, \pi) - \ell(x, \varnothing) \in [-1, 1]$.

Definition 3.1 (Selective transfer risk). For a cell $c = (S, s)$ and a distribution D over the contexts of scope s, $\mathrm{STR}_D(c) = \mathbb{E}_{x \sim D}\big[\ell(x, \pi) - \ell(x, \varnothing)\big]$. With P_s the distribution trials are drawn from and Q_s the deployment distribution, a cell is *admissible* if $\mathrm{STR}_P(c) \le -\theta$ for a benefit margin $\theta > 0$, and

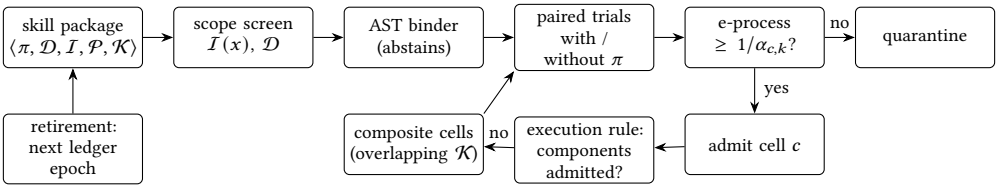


Fig. 1. Gatekeeper. "No" at the e-process means the trial budget was spent. Evidence comes from paired trials on the target scope; each cell (single skill or composite) has its own entry in a pre-allocated error ledger; overlapping skills run together only when their composite cell is admitted; a fingerprint change retires an admission and re-admission draws the next ledger entry.

harmful if $\mathrm{STR}_Q(c) > 0$. After a retirement (Section 5) the environment may have changed; epoch k of a cell then has its own $P_s^{(k)}, Q_s^{(k)}$, and every statement below is per epoch.

The definition is an excess over the no-skill baseline, so a skill that neither helps nor harms is not admissible, and it is relative to a declared scope; Proposition 5.4 shows why it cannot be otherwise.

Assumptions.

Assumption 1 (Paired trials). *For every cell c there is an infinite sequence of tasks $x_1^c, x_2^c, \ldots$ drawn i.i.d. from P_s (in practice, drawn* with replacement *from the target's task set; without replacement the conditional mean of the next draw drifts and Lemma 5.1 needs a finite-population e-process instead); each task is run with and without the skill on the same snapshot and seed. Different cells may depend on each other.*

Assumption 2 (Bounded shift). *Deployment contexts of scope s follow Q_s with* $\mathrm{TV}(P_s, Q_s) \leq \xi_s$, *where* $\mathrm{TV}(P, Q) = \sup_A |P(A) - Q(A)|$.

Execution rule. Skills that are executed together form a set E. Split E into the connected components of its footprint-overlap graph (two skills are adjacent when their footprints intersect, or when applying one creates or removes sites of the other, which we detect by applying both in either order). We execute E only if every component with two or more skills is itself an admitted cell; a skill whose composite with an already admitted skill is still undecided is not executed together with it. This closes the gap between per-cell certificates and what actually runs.

Admission problem. Given a library, a validation budget and a pre-allocated error budget α, decide which cells may execute, such that $\Pr[\exists \text{ an admitted cell } c : \mathrm{STR}_Q(c) > 0] \leq \alpha$ over the whole library, over adaptively chosen cells and over re-admissions after retirement, and such that, under the execution rule, no executed set is harmful except on that event. Theorem 5.2 and Corollary 5.6 give this whenever $2\xi_s \leq \theta$ for every scope; for coarser scopes they give the weaker, explicit statement $\mathrm{STR}_Q(c) \leq -\theta + 2\xi_s$.

4 Approach: Gatekeeper

Figure 1 summarises the pipeline. For a skill S and a target context, Gatekeeper proceeds as follows.

(1) **Scope screen.** Evaluate the input assumptions $\mathcal{I}$ and compare the fingerprints $\mathcal{D}$; missing information means OutOfScope. A pass is necessary, not sufficient: the motivating example passes it.
(2) **Bind.** Apply π with an AST binder that abstains on ambiguity (no match, several matches, an existing guard, a shadowed built-in, a missing symbol) instead of guessing.

(3) **Paired trials.** Draw tasks of the target scope with replacement; run each with and without the skill on the same snapshot and seed, in a fresh worker process; record $Z = \ell(x, \pi) - \ell(x, \varnothing)$.
(4) **Certify.** Update the betting e-process of the cell (Lemma 5.1); admit when it reaches $1/\alpha_{c,k}$, the cell's entry in the pre-allocated ledger; quarantine when the budget is spent. A harm alarm (an e-process for a high harm rate) rejects early when harm is frequent.
(5) **Compositions.** When an admitted skill's footprint overlaps another's, the composite is enqueued as its own cell; until it is admitted, the execution rule keeps the two from running together.
(6) **Retirement.** When the fingerprint of an admitted scope changes, admission is revoked; re-admission draws the next entry of the cell's ledger ($6/(\pi^2(k+1)^2)$ of its share at epoch k), so revocations cannot inflate the error budget.

Capability approval is consumed as a precondition and never merged into the functional decision; the two verdicts and their evidence are stored separately.

5 Theory

We claim no new statistics: the guarantee is assembled from betting e-processes [20], Ville's inequality [13] and a union bound. What is new is the estimand, the per-cell ledger that covers compositions and re-admission, the proof that the shift term is unavoidable, and the exact characterisation of composition risk.

5.1 Per-cell evidence

Lemma 5.1 (Betting e-process). *Let $Y_1, Y_2, \ldots$ take values in $[y_{\min}, y_{\max}]$ with $y_{\min} < \theta$, and suppose $\mathbb{E}[Y_i \mid \mathcal{F}_{i-1}] \le \theta$ for all i (the null H_0). The process $K_0 = 1$, $K_n = \prod_{i \le n}\big(1 + \lambda_i(Y_i - \theta)\big)$ with predictable $\lambda_i \in [0, 1/(\theta - y_{\min}))$ satisfies $\Pr[\exists n : K_n \ge 1/\beta] \le \beta$ for every $\beta \in (0,1)$. For admission, $Y_i = -Z_i \in [-1, 1]$ and H_0^c is $\mathbb{E}[Y] \le \theta$ (the cell is not admissible), so $\lambda_i < 1/(1+\theta)$.*

Proof. Since $Y_i \ge y_{\min}$, every factor is at least $1 - \lambda_i(\theta - y_{\min}) > 0$. By the null and predictability, $\mathbb{E}[K_i \mid \mathcal{F}_{i-1}] = K_{i-1}\big(1 + \lambda_i(\mathbb{E}[Y_i] - \theta)\big) \le K_{i-1}$, so (K_n) is a nonnegative supermartingale with $K_0 = 1$, and Ville's inequality gives the claim. □

The bound holds at any data-dependent stopping time, including the end of the validation budget, which is why a fixed-sample interval would not do. Under Assumption 1 the trials are i.i.d., so the null on the conditional mean reduces to $\mathbb{E}[Y] \le \theta$.

Remark 1 (Harm certificate used in the experiments). The experiments of Section 7 also use a certificate against *harm*: with $H_i \in \{0, 1\}$ indicating a harmful draw and a tolerated harm rate η, the null is $\Pr[H = 1] \ge \eta$. Setting $Y_i = 1 - H_i \in [0, 1]$ and $\theta = 1 - \eta$ gives factors $1 + \lambda(\eta - H_i)$, that is, $1 + \lambda\eta$ for a clean draw and $1 - c$ for a harmful one when $\lambda = c/(1-\eta)$; the condition $\lambda < 1/(\theta - y_{\min}) = 1/(1-\eta)$ is $c < 1$. Lemma 5.1 and Theorem 5.2 therefore apply unchanged, with "harm rate at least η" in place of "not admissible". With $c = 0.9$ and $\eta = 5\%$ the wealth grows only when the true harm rate is below $\log(1+\lambda\eta)/(\log(1+\lambda\eta) - \log(1-c)) \approx 0.39\eta$, and certifying zero harm in a family of 16 cells at $\alpha = 0.1$ takes $\lceil \log(160)/\log(1+\lambda\eta) \rceil = 110$ clean draws.

5.2 Library-wide admission guarantee

Theorem 5.2 (Admission over a declared scope family). *Let C be the countable set of all cells that could ever be formed, compositions included, and let a ledger $\alpha_{c,k} \ge 0$ be fixed in advance, as a function of the cell's identity, for every cell c and epoch $k \ge 0$ (epoch $k \ge 1$ is the k-th re-admission after retirement), with $\sum_{c \in C} \sum_{k \ge 0} \alpha_{c,k} \le \alpha$; for instance $\alpha_{c,k} = \alpha_c \cdot 6/(\pi^2(k+1)^2)$ with $\sum_c \alpha_c \le \alpha$.*

Our implementation fixes α_c by the cell's size alone for a declared library capacity N: $\alpha s/N$ for a single skill and $\alpha(1-s)\,6/(\pi^2(m-1)^2)/\binom{N}{m}$ for a composite of $m \geq 2$ skills, which sums to at most α and does not change as skills are added. A library that grows is covered as long as the share of every future cell is fixed by the same rule. Suppose each epoch uses fresh tasks and admits c only when its e-process (Lemma 5.1) reaches $1/\alpha_{c,k}$, while the choice of cells, their order and the stopping times may depend on all data. Under Assumptions 1 and 2,

$$\Pr\left[\exists \text{ admitted } (c,k):\ \mathrm{STR}_{Q^{(k)}}(c) > -\theta + 2\xi_{s(c)}\right] \leq \alpha,$$

where $P^{(k)}, Q^{(k)}$ are the trial and deployment distributions of epoch k. In particular, if $2\xi_s \leq \theta$ for every scope, the probability of admitting any harmful cell is at most α.

Proof. For every (c,k) let $A_{c,k} = \{\sup_n K_n^{c,k} \geq 1/\alpha_{c,k}\}$, defined on the whole infinite task sequence of that epoch. An admission of (c,k) implies $A_{c,k}$; adaptive selection and stopping can only forgo admissions. Hence the event that some cell with H_0^c true is admitted is contained in $\bigcup_{(c,k):H_0^c} A_{c,k}$, whose probability is at most $\sum \alpha_{c,k} \leq \alpha$ by Lemma 5.1 and a union bound, which needs no independence across cells. Finally, by Lemma 5.3, $\mathrm{STR}_{Q^{(k)}}(c) > -\theta + 2\xi_s$ implies $\mathrm{STR}_{P^{(k)}}(c) > -\theta$, that is, H_0^c for that epoch. If $2\xi_s \leq \theta$, then $\mathrm{STR}_Q(c) > 0$ implies $\mathrm{STR}_Q(c) > -\theta + 2\xi_s$. □

Remark 2 (Fixed families: e-Holm, as used in the experiments). For a fixed family of K cells at epoch 0 the experiments use, instead of thresholding each cell at $1/\alpha_c$, the closed e-Bonferroni (e-Holm) procedure [15] at level α over $e_c = \min(K^{\mathrm{ben}}_{c,\tau_c}, K^{\mathrm{cert}}_{c,\tau_c})$, the benefit e-process and the harm certificate read at the cell's bounded stopping time τ_c. Each K_τ is an e-value by optional stopping ($\mathbb{E}K_\tau \leq K_0 = 1$), the minimum is an e-value under the union null "not admissible or harm rate at least η", and e-Holm controls the family-wise error under arbitrary dependence while admitting every cell that the ledger with $\alpha_c = \alpha/K$ would. Note that e_c is the value *at* τ_c, not the running maximum, which is not an e-value.

Remark 3 (Data-dependent budgets). The ledger above is fixed in advance, which is what makes the union bound free of any independence assumption. A budget that depends on data (for example, α_c proportional to an estimated reuse frequency) remains valid if (i) the total is at most α almost surely and (ii) the trial sequence of cell c is independent of the information used to set α_c: then $\Pr[A_{c,k} \mid \alpha_{c,k}] \leq \alpha_{c,k}$ and summing the expectations gives α. Without (ii), for instance when two cells share tasks, a data-dependent budget can break the guarantee.

Lemma 5.3 (Shift). *If f takes values in an interval of length b, then $|\mathbb{E}_Q f - \mathbb{E}_P f| \leq b \cdot \mathrm{TV}(P,Q)$. Since $Z \in [-1,1]$, $|\mathrm{STR}_Q(c) - \mathrm{STR}_P(c)| \leq 2\,\mathrm{TV}(P_s, Q_s) \leq 2\xi_s$, and the constant 2 is attained.*

Proof. Centre f so that $|g| \leq b/2$; then $|\mathbb{E}_Q g - \mathbb{E}_P g| \leq (b/2)\sum_x |Q(x) - P(x)| = b\,\mathrm{TV}(P,Q)$. For attainment, take task types u with $Z = 1$ and d with $Z = -1$, $P(u) = 0.1$, $Q(u) = 0.2$: then TV $= 0.1$ and the risks differ by 0.2. □

Why the budget is per cell. Allocating one α_s per *scope* and letting every skill in the scope use the threshold $1/\alpha_s$ does not control the family-wise error: with ten skills at the null boundary, $\theta = \alpha = 0.1$ and 300 trials, the family-wise false admission rate of our e-process is 0.45, against 0.05 with the per-cell budget $\alpha/10$ (Section 5.6).

5.3 Scope and shift are necessary

Assumption 3 (Rich target class). *For every value of the features a procedure observes (fingerprints, predicates, trial outcomes on the observed repositories), there is a repository with those features whose task losses are arbitrary in $[0,1]$.*

Proposition 5.4 (No guarantee without target evidence or bounded shift). (a) *Let a procedure decide from data of observed repositories $\mathcal{R}_{\text{obs}}$ only, and admit a skill S with positive probability. Under Assumption 3 there is a repository $R^\dagger \notin \mathcal{R}_{\text{obs}}$ with $\text{STR}(S, \{R^\dagger\}) = 1$ on which the admission probability is unchanged.* (b) *For every $\theta \in (0,1)$, $\delta > 0$ with $\theta + \delta < 1$, $\xi \in (0, 1/2]$ and every procedure, there are P, Q with $\text{TV}(P, Q) = \xi$ and a cell c with $\text{STR}_P(c) = -\theta - \delta$ and $\text{STR}_Q(c) = \text{STR}_P(c) + 2\xi$, whose admission probability does not depend on Q.*

Proof. (a) By Assumption 3, take $R^\dagger$ that has the same values of every feature the procedure reads (the data of $\mathcal{R}_{\text{obs}}$ and the target's own fingerprints and predicates) and has $\ell(x, \pi) = 1$, $\ell(x, \varnothing) = 0$ for every task. The procedure's input, hence its admission probability, is unchanged, and $Z \equiv 1$ on $R^\dagger$. (b) Take task types u ($Z = 1$) and d ($Z = -1$) with $P(d) = (1 + \theta + \delta)/2 \geq \xi$, and obtain Q by moving mass ξ from d to u. The procedure only reads tasks drawn from P. □

Part (b) shows that no procedure that learns only from P can certify Q-risk better than $\text{STR}_P + 2\xi$, so the shift term of Theorem 5.2 cannot be removed or reduced; this is why the guarantee is relative to a declared scope family and why coarse scopes make it weak.

5.4 Composition

Theorem 5.5 (Composition risk is not controlled by individual risks). *For $a, b \in [-1, 0)$,*

$$\sup\{\text{STR}(S_1 \circ S_2) : \ \text{STR}(S_1) = a, \ \text{STR}(S_2) = b\} = 1 + \min(a, b),$$

and the supremum is attained. In particular, two skills that each meet the margin $-\theta$ can compose to excess risk $1 - \theta > 0$, and no bound on the composite risk improves on the trivial one implied by $\ell \in [0, 1]$.

Proof. Upper bound, for any skills: $\text{STR}(S_1 \circ S_2) \leq 1 - \mathbb{E}\ell(\varnothing)$, and $\mathbb{E}\ell(\varnothing) \geq -a$ because $\mathbb{E}\ell(\pi_1) = \mathbb{E}\ell(\varnothing) + a \geq 0$; likewise $\geq -b$. Attainment, with losses realizable by Assumption 3: constant losses $\ell(\varnothing) = -\min(a, b)$, $\ell(\pi_1) = \ell(\varnothing) + a$, $\ell(\pi_2) = \ell(\varnothing) + b$, $\ell(\pi_1 \circ \pi_2) = 1$. □

Theorem 5.5 says nothing about footprints. Screening only pairs whose footprints overlap is therefore sound only under an additional assumption, which we state and test rather than presume.

Assumption 4 (Non-interaction of disjoint footprints). *For sets of skills A, B whose footprints are disjoint ($\mathcal{K}_A \cap \mathcal{K}_B = \varnothing$, with $\mathcal{K}_A$ the union over A), and every task x, $\ell(x, \pi_A \circ \pi_B) - \ell(x, \varnothing) = [\ell(x, \pi_A) - \ell(x, \varnothing)] + [\ell(x, \pi_B) - \ell(x, \varnothing)]$.*

Corollary 5.6 (What actually runs). *Under Assumption 4 and the execution rule of Section 3, outside the event of Theorem 5.2 (probability at most α), every executed set E with m footprint-connected components satisfies $\text{STR}_Q(E) \leq m(-\theta + 2\xi_s)$; in particular no executed set is harmful when $2\xi_s \leq \theta$. Only footprint-overlapping components need a composite cell: with mean overlap degree $\bar{d}$ there are $|\mathcal{L}_s|\,\bar{d}/2$ overlapping pairs per scope.*

Proof. Components have pairwise disjoint footprints, so applying Assumption 4 $m - 1$ times and taking expectations under Q gives $\text{STR}_Q(E) = \sum_{j=1}^{m} \text{STR}_Q(E_j)$. Each E_j is an admitted cell (a single skill, or a composite by the execution rule), so outside the event of Theorem 5.2 each term is at most $-\theta + 2\xi_s$. □

Assumption 4 is strong: two edits in different functions can still interact through shared state that no footprint records, and footprints read off the original code miss a change that creates sites for another: Study R2 found such a pair on real code (Section 7.2), which is why the execution rule also links non-commuting pairs. The assumption is then needed only for pairs that commute and share no function; among the 80 such file pairs of Study R2 none interacted syntactically, but a syntactic check cannot exclude interaction through shared state, so it remains an assumption.

5.5 The value of pairing

Proposition 5.7 (Pairing). *Let* $V_\pi = \mathrm{Var}\,\ell(x,\pi)$, $V_\varnothing = \mathrm{Var}\,\ell(x,\varnothing)$ *and* $C = \mathrm{Cov}(\ell(x,\pi),\ell(x,\varnothing))$ *on the same task. Estimating* STR *from n paired tasks has variance* $(V_\pi + V_\varnothing - 2C)/n$*; from two independent arms of n tasks each it has variance* $(V_\pi + V_\varnothing)/n$*. Both designs cost* $2n$ *runs, so pairing reduces the number of tasks needed for a given precision by* $R = (V_\pi + V_\varnothing)/(V_\pi + V_\varnothing - 2C)$*. Under the additive model* $\ell(\varnothing) = T$, $\ell(\pi) = T + S$ *with independent* T, S, $R = 1 + 2\sigma_T^2/\sigma_S^2$.

Proof. Direct computation of the two variances; $V_\pi, V_\varnothing$ include any run-to-run noise, and only the shared task component enters C. Under the additive model $V_\pi + V_\varnothing = 2\sigma_T^2 + \sigma_S^2$ and $C = \sigma_T^2$. □

5.6 Numerical checks

Every statement above is checked numerically by a script in the artifact, using the e-process of our implementation: per-cell validity, the failure of a per-scope budget, the attained constant 2 in Lemma 5.3, the bound of Theorem 5.5 on random instances, the pairing ratio, the convergence of the ledger, and the failure of re-admission without a ledger (re-testing a retired cell with the full α at every epoch), the harm certificate of Remark 1, the library ledger's total, and the exact decision rule of Studies D–H (Remark 2) at both null boundaries, where its family-wise false admission rate was 2.4% and 7.3% (16 cells, $\alpha = 0.1$).

6 Implementation

Gatekeeper is about 4,800 lines of Python using only the standard library: the pipeline (screen, binder, paired-trial harness, e-processes, ledger, baselines B2, B6-R and B8, retirement) and the extensions used in the studies (function-level oracles, gates, the contract classifier and the execution rule). Paired trials run in per-draw worker processes started from a common snapshot; re-running 2,400 recorded draws in shuffled order with fresh workers reproduced every outcome. The loss is one minus the held-out pass rate plus 0.5 per regression, clipped at 1. Decisions consume the per-test paired outcomes and the labels the set of new failures; the regression penalty only enters the reported suite loss. Recomputing all 7,200 recorded draws of the skill studies and re-running the demonstration suites with penalties from 0 to 5 changed no decision and no label (a one-test suite's clipped loss does not depend on the penalty at all). The execution rule uses module-qualified footprints (file and function, module-level code included) and the enabling edges of Section 7.2. A conformance check labels each configuration with the guarantee it actually has: options of the frozen pipeline such as consuming priority tasks first, keeping only tasks that reach the edit, a sign test on discordant pairs, or e-BH give a weaker guarantee than Theorem 5.2 (no guarantee, sign only, sign only, FDR), and no reported study uses them. Every theorem and remark is checked numerically on the code that the studies ran (Section 5.6), including the exact decision rule of Studies D–H.

Every study is pre-registered by a workflow that hashes its protocol, configuration and tools; the seed is drawn from the public drand beacon after the freeze; amendments re-hash the study and keep the seed. Each result is produced by a workflow and committed with its raw records; a checker recomputes every number quoted in this paper from those records, and a separate script checks every theoretical statement numerically. The test suite has more than 350 tests.

7 Evaluation

We ask four questions. **RQ1** Does the certificate behave as the theory predicts, where do simple gates fail, and does the library-level guarantee hold for what actually runs? (Sections 7.1 and 7.2) **RQ2** On real skill reuse, how does admission compare with replay plus fingerprints and with always-retest? (Section 7.3) **RQ3** Does partial harm occur in plausible LLM patches, and which

Table 1. Probability that a gate admits an always-helpful patch, by true harm rate h ($h \geq \eta$: admitting is an error).

h	smoke 20	zero-tol. 300	fixed 300	certificate, 300	mixture, 1000
0.3%	0.94	0.41	1.00	1.00	1.00
1%	0.82	0.05	1.00	0.95	1.00
2%	0.67	0.00	0.92	0.65	1.00
3%	0.54	0.00	0.59	0.36	0.78
5% (error)	0.36	0.00	0.06	0.10	0.10
10% (error)	0.12	0.00	0.00	0.01	0.00

gates catch it? (Section 7.4) **RQ4** How much of the measured harm depends on how harm is defined? (Sections 7.5 and 7.6)

All studies were pre-registered: protocol, configuration and tools were hashed before any outcome was seen, seeds were drawn from the public drand beacon after the freeze, and amendments are recorded and labelled. Results that were analysed after seeing data are marked as such. Every number below is produced by a workflow in the artifact.

7.1 Where simple gates fail: the phase diagram

For a patch that always helps, Table 1 gives the probability that each gate admits it as a function of the true harm rate h ($\eta = 5\%$, $\alpha = 0.1$; exact for the smoke, zero-tolerance and fixed-sample gates, Monte Carlo for the sequential rules). A zero-tolerance gate with 300 draws keeps fewer than half of the acceptable patches once $h > 0.23\%$; a 20-draw smoke test still admits more than 10% of inadmissible patches until $h \approx 11\%$. Outside the band $[3/n,\ 1 - 0.1^{1/20}] \approx [0.75\%, 10.9\%]$ ($n = 400$) a simple gate suffices. At the same number of draws a fixed-sample test is as safe as the sequential certificate and *more* powerful (0.92 versus 0.65 at $h = 2\%$, 300 draws). What the sequential rule buys is validity at data-dependent stopping and early stopping (110 clean draws in a 16-cell family), not power. On synthetic targets with rare harm (Study H, 50 repetitions per cell, 300 draws), the certificate admitted 0/100 cells with $\rho \geq \eta$, while a 5-draw smoke test admitted 38/50 and 27/50 at $\rho = 5\%$ and 10%, and replay plus fingerprints admitted every cell.

7.2 Library-level control (simulation)

The libraries of Studies D–G have no overlapping skills, so we first test the library-level parts of the design (composite cells, the per-cell ledger and the execution rule) on synthetic libraries built with the implementation's own e-process, ledger and execution rule. Each library has 12 skills in one scope (70% beneficial, STR = −0.3; the rest not admissible), each touching one of 8 code regions; every overlapping pair is, with probability q, the harmful construction of Theorem 5.5 (STR = +0.5 when applied together), and risks otherwise add (1,000 libraries, 300 trials per cell, $\theta = \alpha = 0.1$). Admitting skills one by one and running every admitted skill together executed a harmful footprint component in 27.9% of the libraries at $q = 0.3$ (9.9% at $q = 0.1$, 42.4% at $q = 0.5$). With composite cells, the ledger and the execution rule this never happened (0/1,000 at every q, consistent with Corollary 5.6). The price is reuse: 3.59 instead of 4.48 beneficial skills executed per library, because undecided or rejected composites keep overlapping skills apart, and 8% more trials. Measured on the executed set as a whole, one-by-one admission looked harmful in only 5.4% of libraries: benefits elsewhere masked the harmful component, which is why the guarantee is stated per component.

A twelve-family library on real code (Study R2). To see what the library-level rules meet on real code, we built a library of twelve families of reusable code changes: the two frozen families of

Table 2. The Study R2 library: families, the inputs on which each rewrite changes behaviour, and the number of the 16 target repositories where it binds. The first two are the frozen families of Studies D–G.

Rewrite	Changes behaviour when	Repos
`isinstance(x, int)` → … `and not isinstance(x, bool)`	x is a `bool`	15
`x == None` → `x is None`	`__eq__` overridden	3
`type(x) == T` → `isinstance(x, T)`	x is an instance of a subclass	4
`x == True` → `x is True`	x is 1, a numpy bool	2
`x is 1000` → `x == 1000`	equal, non-identical values	0
`len(x) == 0` → `not x`	`__bool__` and `__len__` disagree	8
`k in d.keys()` → `k in d`	`__contains__` and `keys()` disagree	3
`def f(a=[])` → `a=None` + guard	callers rely on shared state	5
`except:` → `except Exception:`	`KeyboardInterrupt`, `SystemExit`	5
`raise e` → `raise` (in its handler)	the traceback is inspected	4
`open(p)` → `open(p, encoding="utf-8")`	non-UTF-8 locale or file	10
`yaml.load(s)` → `yaml.safe_load(s)`	Python-specific tags	0

Section 7.3 and ten linter-style rewrites (Table 2), each behaviour-preserving on ordinary inputs and behaviour-changing on some (every family has a unit test with such a witness input). A pre-registered census (AST analysis only) over all 16 target repositories at their pinned revisions (655 non-test files) found 59 single-skill cells, 3.7 per repository, touching 467 functions, of which 21 are touched by two or more families. Of the 136 family pairs that meet in some repository, 18 overlap, and the execution rule requires 6 composite cells. Applying every two families that bind in the same file in both orders, all 21 such pairs that share a function commute. Two pairs with *disjoint* footprints do not: rewriting `type(x) is not int` to `not isinstance(x, int)` admits `bool` and creates a site for the `bool` guard in a function the guard did not touch, so the result depends on the order. In `pydal` the original-code footprints put the two families in different components, and the execution rule would have run them together without a composite certificate. Footprints taken from the original code therefore miss *enabling* interactions; the implementation now also links every non-commuting or site-changing pair (post hoc amendment R2-A1), which merges the two components. Re-running the simulation on the 13 measured libraries with two or more cells (enabling edges included; benefits and harms synthetic as above), one-by-one admission executed a harmful component in 7.1% of libraries at $q = 0.3$ (14.5% when every overlapping pair is harmful), ours in 0/1,000, at the cost of 1.64 instead of 1.96 beneficial skills executed. Real libraries overlap less than the synthetic ones, so both the risk of one-by-one admission and the cost of the rule are smaller; neither vanishes.

7.3 Skill reuse on real targets

Two skill families, both frozen AST rewrites (excluding `bool` from `int` checks, learned from one PySStuBs fix, and `== None`→`is None`), were applied to 14 real targets in three pre-registered studies (28 cells). *Study D* (8 targets, 16 cells): ground truth from 300 draws plus the target's full test suite gave 3 beneficial, 9 unchanged, 4 out-of-scope and 0 harmful cells. B2, replay plus fingerprints (B6-R) and always-retest (B8) each admitted 12 cells, the 3 beneficial ones and all 9 unchanged ones; the certificate (the smaller of the benefit e-process and the harm certificate where the cell stopped, with e-Holm over the 16 cells, Remark 2, family-wise $\alpha = 0.1$) admitted 2 cells, both beneficial, after 110 draws each, and missed the third, whose mean benefit (-0.11) is smaller than the margin $\theta = 0.2$. It used 3,220 trial runs against 3,600 for B8. *Studies E and G*: on the four harmful cells (`pyyaml`, `packaging`, `databricks`: harm on 87–100% of inputs; `pydal`: 81/300), B2 and B6-R admitted all four and the certificate rejected all four, within two draws for the first three. The targets' own test

Table 3. Cells admitted by each comparator on real skill-reuse targets, by ground truth from 300 draws plus the target's test suite. Study G's second cell (no change in draws, a census regression) is omitted.

Comparator	Study D beneficial (of 3)	Study D unchanged (of 9)	Study E unchanged (of 7)	Studies E, G harmful (of 4)
B2 unrestricted reuse	3	9	7	4
B6-R replay + fingerprints	3	9	7	4
B8 always-retest	3	9	7	0
Gatekeeper (certificate)	2	0	0	0

suites (1,287, 51,524 and 130 tests) showed *no* new regression in Study E; in Study G they did (21 new failures). Re-running 2,400 recorded draws in shuffled order with fresh workers reproduced every paired outcome, so the draws behave as independent, as Assumption 1 requires. Table 3 summarises the decisions.

Reading. On these targets harm was frequent and deterministic, so a simple gate with target evidence suffices; the certificate's distinctive behaviour is refusing changes that are not beneficial, not catching rare harm.

7.4 Partial harm in LLM patches

Study P drew fresh repository changes from a pre-registered frame of 3,562 candidates; for each usable instance a light model generated one patch, and 205 patches were plausible (they pass the hidden tests). With a function-level oracle (400 inputs per patch), 16/205 = 7.8% (Wilson 4.9–12.3%) had an estimated regression rate inside the partial-harm band. The test-based census admitted all 16 patches with $\hat{\theta} > 5\%$, a 20-draw smoke test admitted 2, and the zero-tolerance, fixed-sample and sequential gates admitted none. A pre-registered manual check (RQ5) then found that 97.5% of the "regression" inputs were inputs the developers' fix also rejects; counting only regressions on inputs the fix accepts left 1 of 205 patches as planned, 3 when deliberate rejections that the patch skips are added (after seeing data), and at most 6 when non-reproducible patches are counted.

7.5 Naive differences versus contract regressions on SWE-bench Verified

Study Q tested the RQ5 finding on an independent, widely used benchmark. For 469 SWE-bench Verified instances with at least one public resolved patch, up to three distinct resolved patches were scored in the official Docker images (245 instances usable, 694 patches). 43.1% of patches behaved differently from the developers' fix on some input; 11.1% (Wilson 9.0–13.6%) had a *contract regression*, an input on which the fix returns a value and the patch does not, or the fix rejects on purpose and the patch accepts. The confirmatory hypothesis, that most naive differences are not contract regressions, was *not* supported: 6/55 (10.9%, Wilson 5.1–21.8%; one-sided $p \approx 1$). A manual review added after the result (labelled as such) re-ran the 55 units with outputs kept: 36 reproduced exactly, and two independent raters agreed on 87.7% of 154 items. 8/36 (22.2%, Wilson 11.7–38.1%) units had no regression confirmed by both raters; excluding three units whose inputs are model-built stand-in objects gives 11/36 (18.0–46.9%). The result does not hinge on one repository (post hoc): leaving out any one of the twelve repositories keeps the HQ1 share between 8.2% and 16.7% (largest Wilson upper bound 31.9%, without `sympy`) and the contract-regression share between 8.2% and 14.0%, although Django contributes 215 of the 469 instances. The RQ5 conclusion therefore does not generalise: on SWE-bench Verified most naive differences are real, observable behaviour differences from the fix. Whether they are *wrong* is a separate question that a reference implementation cannot answer. The orders of magnitude agree with PatchDiff [19],

which found 29.6% of plausible SWE-bench Verified patches behaving differently from the fix with differential tests and judged 28.6% of those certainly incorrect; our function-level inputs find more differences (43.1%), and our contract classification is a cheaper, automatic proxy for their manual judgement.

7.6 Who defines harm

On 14 measurable targets of Studies D, E and G, a model-written input generator agreed with the authors' hand-written oracles on 11/14 detection decisions (Cohen's $\kappa = 0.57$), a signature-based generator on 8/14 ($\kappa = 0.19$). In one disagreement the generated inputs were right: on `cerberus` the skill rewrote two functions other than the intended one; the hand-written oracle, which checks the intended region, saw nothing, while generated inputs changed behaviour on 24–56 of 200 inputs. An oracle written from the author's intent misses side effects outside that intent, which is the scope problem this paper is about.

8 Threats to Validity

Assumptions of the guarantee. Theorem 5.2 assumes i.i.d. trials drawn with replacement (Assumption 1); we checked that the harness is stateless (2,400 re-run draws, 0 differences) but not that real task streams are identically distributed. The shift bound ξ_s (Assumption 2) is declared, not estimated; the guarantee excludes harmful admissions only when $2\xi_s \leq \theta$, and Proposition 5.4 shows that no procedure learning from trials alone can do better. Screening only footprint-overlapping pairs relies on Assumption 4, which *we have not tested*: two edits in different functions can still interact through state no footprint records. The execution rule is a design constraint that a deployment must enforce.

Construct validity. "Harm" is a difference from a reference: a hand-written oracle (Studies D–H), or the developers' fix (Studies P, Q). A difference from the fix is not necessarily a defect: in the manual review some patches arguably behaved better than the fix. Classifying an input as valid because the fix returns a value overestimates contract regressions when generated inputs are stand-in objects (three of 31 reviewed units); the sensitivity analysis keeps the conclusion. Hand-written oracles check the author's intent and miss side effects outside it (`cerberus`).

Internal validity. The second rater shares a model family with the input generator, and sees outputs but not inputs. 19/55 re-run units did not reproduce because the model-written generator changed; they enter as bounds. The model-written generator sees the fix's diff, so inputs concentrate near the changed region. Four Study Q instances failed for a tool limitation (patches that create files) and count as unusable. The Study Q analysis script defined the inflation ratio differently from the protocol; we report both.

Simulation. The library-level results (Section 7.2) come from synthetic libraries whose interaction model satisfies Assumption 4 by construction; they show what the design guarantees and costs under that model, not how often real skills interact. The library census (Study R2) covers twelve rule-based families on 16 repositories and detects interaction only syntactically (order dependence and created or removed sites); larger or learned libraries concentrate on popular code and may overlap and interact more. The enabling-edge rule was added after the census had found the case (amendment R2-A1).

External validity. The admission studies cover two hand-written rewrite families on 14 targets (the library census, twelve); we make no claim about learned skill libraries at scale, compositions in the wild, or retirement beyond a constructed demonstration. In Study P half of the in-band

cases come from repositories that look like student assignments; Study Q covers public resolved submissions to one benchmark.

9 Related Work

9.1 Skill libraries and experience reuse

Voyager established the skill-library pattern [16]. GSE learns globally reusable skills for coding agents, models inter-skill relations, consolidates skills and guards them with replay verification [21]; it is the closest work and our replay-plus-fingerprint baseline (B6-R) is a reconstruction of that kind of guard. Replay asks whether a skill still works on its source; admission asks whether it should run on this target, which only target evidence answers. Structured experience memories for code agents [5, 17] reuse development experience and face the same scope question.

9.2 Capability and authority

Formal verification of agent skills [9] and privilege control for agents [4, 11] decide what a change may touch. That predicate is different from whether the change is functionally right for the target, and we keep the two decisions apart.

9.3 Patch correctness

Plausible patches that pass the tests can still be wrong: automated repair overfits weak test suites [12], and on SWE-bench Verified [7] PatchDiff exposed behavioural differences from the developers' fix in 29.6% of plausible patches [19]. We measure the same kind of difference with function-level inputs and show that how "harm" is defined (any difference versus a contract regression) changes the measured rate by a factor of about four.

9.4 Sequential testing in deployment

Always-valid inference lets online experiments be monitored continuously without inflating error [8]. Admission is the same kind of decision applied to a reusable code change, with three differences we address: the decision is relative to a declared scope that may shift (Proposition 5.4), it must hold over a growing library with re-admission, and compositions are decisions of their own (Theorem 5.5).

9.5 Risk control, e-processes and multiple testing

Conformal prediction [14], conformal risk control [2] and Learn-Then-Test [1] give distribution-free guarantees under exchangeability. Skill transfer violates exchangeability across repositories, so our guarantee is relative to a declared scope family (Proposition 5.4). For sequential evidence we use betting e-processes [20] and Ville's inequality [13], which make an admission valid at any data-dependent stopping time [6, 10]. Simultaneous control over the library is family-wise (FWER) and comes from a union bound over cells; it requires no independence. A false-discovery-rate variant is available through e-BH [18], at the price of a weaker, FDR-type guarantee; Benjamini–Hochberg [3] is used only for the secondary comparisons of the evaluation.

10 Conclusion

A reusable code change can be authorized, pass its replay tests and still be wrong for the repository it is about to run on, and two changes that are each beneficial can be jointly harmful. We gave an admission rule whose error budget holds over a whole library, over compositions and over re-admission; proved that its shift term cannot be removed and that individual risks say nothing useful about composite risk; and tied certificates to what actually runs through an execution rule.

Pre-registered experiments, negative results included, show where such a gate matters and where it does not: on our real skill targets harm was frequent and a simple gate with target evidence sufficed, the certificate's distinctive behaviour being its refusal of changes that do not help; in LLM patches partial harm exists but depends strongly on how harm is defined.

Data Availability

The artifact contains the implementation, every study's frozen protocol and hashes, the workflows that produced each result, the raw per-instance records, the number checker for this paper and the numerical checks of the theory.

References

[1] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. 2021. Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control. arXiv:2110.01052.

[2] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal Risk Control. In *International Conference on Learning Representations (ICLR)*.

[3] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society Series B* 57, 1 (1995), 289–300.

[4] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813.

[5] Yi-Xuan Deng, Xiaoqin Liu, Yi Zhang, Guo-Wei Yang, and Shuojin Yang. 2026. Your Code Agent Can Grow Alongside You with Structured Memory. arXiv:2603.13258.

[6] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, and Jasjeet Sekhon. 2021. Time-Uniform, Nonparametric, Nonasymptotic Confidence Sequences. *The Annals of Statistics* 49, 2 (2021), 1055–1080.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.

[8] Ramesh Johari, Pete Koomen, Leonid Pekelis, and David Walsh. 2022. Always Valid Inference: Continuous Monitoring of A/B Tests. *Operations Research* 70, 3 (2022), 1806–1821. doi:10.1287/opre.2021.2135

[9] Alfredo Metere. 2026. Methods for Formal Verification of Agent Skills: Three Layers Toward a Mechanically Checkable Capability-Containment Proof. arXiv:2605.23951.

[10] Aaditya Ramdas, Peter Grünwald, Vladimir Vovk, and Glenn Shafer. 2023. Game-Theoretic Statistics and Safe Anytime-Valid Inference. *Statist. Sci.* 38, 4 (2023), 576–601.

[11] Tianneng Shi, Jingxuan He, Zhun Wang, Linyu Wu, Hongwei Li, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703.

[12] Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the Cure Worse than the Disease? Overfitting in Automated Program Repair. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 532–543.

[13] Jean Ville. 1939. *Étude critique de la notion de collectif*. Gauthier-Villars, Paris.

[14] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. 2005. *Algorithmic Learning in a Random World*. Springer.

[15] Vladimir Vovk and Ruodu Wang. 2021. E-values: Calibration, Combination and Applications. *The Annals of Statistics* 49, 3 (2021), 1736–1754. doi:10.1214/20-AOS2020

[16] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research* (2024).

[17] Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, Heng Lian, Kunyi Wang, Xiaoming Yu, Jianghao Yin, Sen Hu, Yue Hu, Shaolei Zhang, Yanbing Liu, Ronghao Chen, and Huacan Wang. 2026. MemGovern: Enhancing Code Agents through Learning from Governed Human Experiences. In *Findings of the Association for Computational Linguistics: EMNLP 2026*. arXiv:2601.06789; ACL Anthology 2026.findings-emnlp.898.

[18] Ruodu Wang and Aaditya Ramdas. 2022. False Discovery Rate Control with E-values. *Journal of the Royal Statistical Society Series B* 84, 3 (2022), 822–852.

[19] You Wang, Michael Pradel, and Zhongxin Liu. 2026. Are "Solved Issues" in SWE-bench Really Solved Correctly? An Empirical Study. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. doi:10.1145/3744916.3764576

[20] Ian Waudby-Smith and Aaditya Ramdas. 2024. Estimating Means of Bounded Random Variables by Betting. *Journal of the Royal Statistical Society Series B* 86, 1 (2024), 1–27.

[21] Chen Yang, Jiashuo Tian, Ziqi Wang, Xinyin Liu, Meiru Ye, and Junjie Chen. 2026. Learning Globally Reusable Skills for Coding Agents. arXiv:2608.06153.